

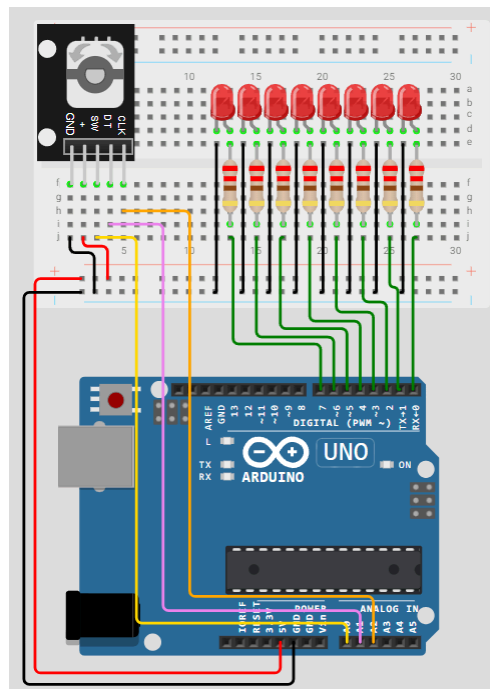
Atividade_02 - Revisão de Conceitos. Livro AVR e Arduino – Técnicas de Projeto Capítulos: 4 (Manipulação de bits) e 5 (Portas)

Título: Conhecendo o Atmega328 e acessando portas de saída usando C/C++ / Debounce de botões e Encoders de posição

Objetivos: Entender a importância do debounce de botões e também o funcionamento de encoders como interface ao microcontrolador.

Nesta prática utilizaremos o Wokwi para simular um circuito simples usando o microcontrolador Atmega328p, utilizado nas placas Arduino UNO. Como entrada será utilizado um encoder rotativo e como saída 8 leds. O circuito base se encontra no link:

<https://wokwi.com/projects/373943840344172545>



Procedimentos:

1. Utilize o circuito fornecido para ativar os leds conforme pedido. Neste circuito os leds estão ligados ao PORTD e o encoder (nossa interface de entrada) ao PORTC. Note que os pinos PC1 e PC2 estão ligados aos pinos DT e CLK do encoder (responsáveis pela codificação de movimento rotatório) e o pino SW (chave central do encoder), ao pino PC0.

2. Para entender como o encoder funciona, visite o site

<https://www.allaboutcircuits.com/projects/how-to-use-a-rotary-encoder-in-a-mcu-based-project/> ou o site

<https://howtomechatronics.com/tutorials/arduino/rotary-encoder-works-use-arduino/>.

3. Programe para que o circuito execute o seguinte: ao girar a chave no sentido anti-horário, os leds devem se acender da **direita para a esquerda**. Ao girar a chave no sentido horário, os leds se apagam da direita para a esquerda, um de cada vez. Ao clicar na chave central, os leds acesos se apagam e os apagados se acendem. Aqui a ideia é utilizar deslocamento de bits.

Notem que os pinos CLK e SW devem ter implementado em suas rotinas o debounce. Já o pino DT não necessita de debounce. Isto ocorre porquê existe uma defasagem entre os pinos CLK e DT em 90 graus. Desta forma, quando ocorre o acionamento da chave ligada ao CLK, a chave ligada ao DT já estará num estado estável, podendo a sua leitura ser efetuada sem qualquer controle temporal, simplesmente lendo o registrador PINC na posição 1.

Vocês podem aproveitar o código de debounce fornecido (aqui).

4. **EXTRA:** Ao segurar o botão central por mais de 900ms, todos os leds se apagam. Durante o intervalo em que o botão é seguro, o microcontrolador deve ficar livre para executar quaisquer outras rotinas.

5. Cole o código fonte do microcontrolador ao final deste arquivo. **Anexe também, junto ao pdf, um arquivo texto com o código fonte.**

ATENÇÃO: Não utilize rotinas de interrupção. Não utilize pinMode, digitalWrite ou digitalRead. O uso destas funções acarretará na avaliação do trabalho com nota nula.

Rúbrica:

Encoder funciona: 30%

Debounce do encoder e sua chave: 30%

Leds fazem o pedido: 40%

Valor desta atividade na média: 0.6

EXTRA: 0.1 na média.

```
long lastClick = 0;
char lastSwState = 1;

int lastClk = HIGH;
unsigned long swPresTime = 0;
bool swPressed = false;
bool longPressHandled = false;

void setup() {
  DDRD = 0xFF;          // Todos os pinos do PORTD como saída
  PORTD = 0x00;         // Todos os LEDs apagados

  DDRC &= ~(1 << PC2); // CLK
  DDRC &= ~(1 << PC1); // DT
  DDRC &= ~(1 << PC0); // SW

  PORTC |= (1 << PC2); // Pull-up
  PORTC |= (1 << PC1);
  PORTC |= (1 << PC0);

}
```

```

void loop() {
  int newClk = (PINC & (1 << PC2)) ? HIGH : LOW;
  int dtValue = (PINC & (1 << PC1)) ? HIGH : LOW;
  int swValue = (PINC & (1 << PC0)) ? HIGH : LOW;

  if (newClk != lastClk) {
    lastClk = newClk;

    if (newClk == LOW) {
      if (dtValue == HIGH) {
        if (PORTD != 0x00) {
          // Apagar da direita para esquerda (bit mais à esquerda aceso)
          for (int i = 0; i < 8; i++) {
            if (PORTD & (1 << i)) {
              PORTD &= ~(1 << i); // Apaga o bit
              break;
            }
          }
        }
      } else {
        if (PORTD == 0x00) {
          PORTD = 0x01;
        } else if ((PORTD & 0x80) == 0) {
          PORTD = (PORTD << 1) | 0x01;
        }
      }
    }
  }
}

if (swValue != lastSwState && (millis() - lastClick) > 20) {
  lastClick = millis();

  if (swValue == LOW) {
    swPressTime = millis();
    swPressed = true;
    longPressHandled = false;
  } else {
    if (swPressed) {
      unsigned long duration = millis() - swPressTime;

```

```
    if (duration < 900) {
        PORTD = ~PORTD;
    } else if (!longPressHandled) {
        PORTD = 0x00;
    }

    swPressed = false;
}

lastSwState = swValue;
}

if (swPressed && !longPressHandled) {
    if ((millis() - swPressTime) >= 900) {
        PORTD = 0x00;
        longPressHandled = true;
    }
}
}
```